Few-Shot Defect Inspection on a Laptop CPU: A Compute-Efficiency Benchmark of Frozen CLIP with Append-Only Memory

Mounika Reddy Ashi Sharma
[Department], [Institution]

Abstract

Small manufacturers need visual inspection that starts from a handful of photos of good parts, runs on hardware they already own, and can take on a new product without disturbing the ones in production. We benchmark a training-free detector for this setting: a frozen OpenCLIP backbone, an append-only per-product memory of patch tokens, multi-scale window scoring and optional fusion with text prompts. On MVTec AD, VisA and NEU-DET, three backbones (ViT-B/32, B/16, L/14), two precisions (FP32, INT8) and 1/5/10 shots over three seeds, we report accuracy and CPU cost together, with every comparison paired. With ten good photos the detector reaches 0.925–0.933 image AUROC on MVTec AD, statistically level with PatchCore given the same photos, which localises better. The cheaper ViT-B/32 matches B/16 when defects are large (MVTec AD, NEU-DET) and loses 0.05–0.09 AUROC when they are smaller than one of its patches (VisA). INT8 costs 0.005–0.02 AUROC and never changes which backbone to choose. Adding a product is bit-identical to building it in from the start and leaves every other product's scores unchanged. A laptop that runs at 37–88 ms per image when cool slows five- to six-fold under continuous use, yet still inspects 10,000 parts a day for under Rs 41 of electricity a year.

1 Introduction

A small manufacturer that wants automated visual inspection meets three obstacles before accuracy is even a question. It has photographs of good parts but almost none of defective ones, because a new line has not failed yet. It has a laptop, not a GPU server or a machine-learning team. And its product mix changes: a new part arrives, and the inspection system must take it on without being retrained and without disturbing the parts already being inspected.

Industrial anomaly detection has largely solved the first obstacle by learning only from normal images [2], and memory-bank methods such as PatchCore [7] reach near-perfect accuracy on standard benchmarks. They do so with hundreds of normal images per product and report accuracy alone. Vision–language models have since brought the number of images down to a handful [5], but they are evaluated on GPUs, and what they cost on the hardware a small manufacturer actually owns is not reported.

This paper measures that cost. We study a deliberately simple, training-free detector: a frozen CLIP image encoder [6, 3], an append-only memory of the patch features of a few good photos per product, multi-scale window scoring, and fusion with text prompts. We report accuracy and CPU cost together, across three backbones, two precisions and one to ten shots, on three public benchmarks, with every comparison paired over three seeds, and against PatchCore given exactly the same photos. Our contributions:

- a CPU-only benchmark of accuracy against latency, memory and FLOPs, with paired statistics and both burst and sustained-load latency;
- append-only product addition, verified to be bit-identical to building the product in from the start and to leave every other product unchanged;
- a rule for choosing a backbone that holds on three datasets: the smallest defect that matters against the image area one patch covers;
- a like-for-like comparison with few-shot PatchCore, and a cost analysis showing that inspection on an existing laptop costs tens of rupees a year;
- an open release with one command per table and a working demo.

2 Related work

Learning from normal images. MVTec AD [2] and VisA [11] established the protocol of training only on defect-free images and testing on both. Embedding-based methods model the distribution of features from an ImageNet-pretrained network: PaDiM [4] fits a Gaussian per patch position, and PatchCore [7] keeps a coreset of patch features and scores a test patch by its distance to the nearest one. Our memory follows Patch-

Core's nearest-neighbour principle but stores every support patch, since a few-shot memory is small enough not to need a coreset, and keeps one memory per product so that products are independent. Anomalib [1] provides the PatchCore implementation we use as a baseline.

Vision–language anomaly detection. CLIP [6] aligns images and text in one embedding space, which makes zero-shot detection possible by comparing an image with prompts for normal and defective objects. WinCLIP [5] adds window-level features for localisation and, with a few normal images, a reference memory (WinCLIP+). AnomalyCLIP [10] learns object-agnostic prompts instead. Both report accuracy on GPU; neither reports CPU latency, quantisation cost, or what happens to existing products when one is added. We borrow WinCLIP's two ideas, window aggregation and fusing text with the memory, but compute windows from a single encoder pass and measure what each idea is worth.

Training-free adaptation. Tip-Adapter [9] adapts CLIP to few-shot classification with a key–value cache of support features and no training. We implemented its cache kernel as an anomaly head and found that at its best setting it reproduces nearest-neighbour scoring exactly, because the kernel is monotonic in cosine similarity and AUROC is rank-based; we report this as a negative result rather than use it.

3 Method

Encoder. A frozen OpenCLIP ViT (OpenAI weights, QuickGELU activation) maps a 224×224 image to a pooled embedding in the 512-dim image–text space and to $H \times W$ patch tokens. We keep patch tokens at the transformer width (768 for B/32 and B/16, 1024 for L/14) rather than projecting them to 512: the unprojected tokens are +0.020 to +0.026 image AUROC better (3.7–4.4× the paired standard error, 15 categories × 3 seeds). No parameter is trained anywhere.

Append-only memory. Adding a product stores the ℓ_2-normalised patch tokens of its k good photos in a bank owned by that product alone. A query patch's anomaly is $1 - \max_j \cos(q, m_j)$ over the bank; the image score is the maximum over patches, and the $H \times W$ map is bilinearly upsampled and Gaussian-smoothed ($\sigma = 4$). Because banks share nothing, adding a product cannot change any other product's scores; we verify this with exact tensor equality rather than assume it (Section 5.1).

Multi-scale windows. Following WinCLIP's observation that defects are judged better over a neighbourhood [5], but without its per-window encoder passes, we average-pool the already computed patch tokens over 2×2 and 3×3 windows, score each scale against an equally pooled bank, spread window scores back over the patches they cover, and average the scales. Scale 1 alone is exactly the memory above.

Text fusion. Unprojected patch tokens cannot be compared with text, but the pooled embedding can. The image score becomes $(1-w)\, s_{\text{memory}} + w\, s_{\text{text}}$, where s_{text} is the softmax weight of a "defective ⟨object⟩" prompt ensemble against a "normal" one; a typed defect description joins the defective ensemble. The map is unaffected. We fix $w = 0.5$, WinCLIP+'s equal weighting, and report the full grid $w \in \{0, 0.1, 0.25, 0.5, 1\}$ rather than select on test data.

Deployment. The visual tower is exported to ONNX (identical to PyTorch on 50 real test images: patch cosine 1.00000, identical anomaly scores) and dynamically quantised to INT8, which needs no calibration set. A Streamlit application adds products from photos and an optional description, inspects parts with a heat map, and keeps products on disk between sessions.

4 Protocol

Datasets. MVTec AD [2] (15 categories) and VisA [11] (12) with their pixel masks. NEU-DET [8] has six classes of steel-surface defect and no defect-free images, so each class in turn plays "normal": support photos come from it, and its test set is its 60 held-out images plus 60 drawn evenly from the other five classes (image AUROC only; no masks).

Splits and statistics. 1/5/10 normal support images, nested ($1 \subset 5 \subset 10$) so extra shots are measured against the same starting images, on a fixed test set, for three seeds (42, 1, 2); all manifests are committed. Comparisons are paired over identical (category, shots, seed) splits and reported as the mean delta and its multiple of the paired standard error (SE); we call an effect established above 2×SE.

Hardware. One Intel i7-13700H laptop (6 performance + 8 efficiency cores), CPU only. Latency is the median of 20 batch-size-1 passes after 5 warm-up passes at 6 threads (Section 5.2 explains why not 14).

5 Results

Backbone choice depends on defect size. On MVTec AD no B/16–B/32 difference reaches 2×SE (at 10 shots: +0.000 image, −0.004 pixel), and on NEU-DET none exceeds 0.4×SE. On VisA, B/16 wins at every shot count (+0.093, +0.050, +0.052 image; 4.4–5.5×SE). The reason is defect size against patch size. A B/32 patch

Table 1: Image / pixel AUROC, prototype memory, mean over 3 seeds. NEU-DET is image AUROC only. (`benchmark` CSVs; paired tests in `visa_backbones.json`, `neu_det_backbones.json`, `*_l14_vs_b16.json`.)

	backbone	1-shot	5-shot	10-shot
MVTec	B/32	.837/.920	.884/.934	.904/.940
	B/16	.856/.916	.888/.932	.904/.936
	L/14	.883/.930	.920/.942	.933/.945
VisA	B/32	.681/.905	.761/.930	.779/.935
	B/16	.773/.934	.811/.947	.831/.951
	L/14	.785/.950	.828/.961	.843/.964
NEU-DET	B/32	.818	.867	.893
	B/16	.821	.868	.887
	L/14	.838	.883	.897

Table 2: Scoring ablations, ViT-B/16, image AUROC. Windows: $\{1, 2, 3\}$. Text: $w = 0.5$. (`text_fusion*.json`, `*_windowed.json`, `neu_det_text.json`.)

	scoring	1-shot	5-shot	10-shot
MVTec	memory	.856	.888	.904
	+ windows	.866	.907	.921
	+ text	.872	.897	.911
	+ both	**.881**	**.913**	**.925**
	text only	.802	.802	.802
VisA	memory	.773	.811	.831
	+ windows	.789	.832	.851
	+ text	.791	.820	.837
	+ both	**.815**	**.841**	**.855**
	text only	.698	.698	.698
NEU-DET	memory	.821	.868	.887
	+ windows	**.865**	**.917**	**.935**
	+ both	.825	.866	.881

covers 2.0% of a 224 px image, a B/16 patch 0.5%. Measured from the masks, a VisA defect covers 0.04–0.9% of the image; an MVTec AD defect averages 4.6%; NEU-DET defect boxes average 39%. B/16's largest VisA gains are its smallest-defect categories. L/14 (0.4% per patch) improves on B/16 again on MVTec AD (+0.030 image, 4.6×SE), most on `toothbrush` (+0.117) and `screw` (+0.086); on VisA its pixel gain is established (+0.013, 3.9×SE) but its image gain is not (+0.012, 1.7×SE), and on NEU-DET neither is (at most 1.1×SE).

More shots buy predictability. On MVTec AD with B/16 the mean seed-to-seed standard deviation of image AUROC falls from 0.027 at one shot to 0.012 at ten; single-seed few-shot numbers understate this variance.

Windows help everywhere; text helps where "normal" means something. Windows add +0.017 to +0.019 image AUROC at 5 and 10 shots on MVTec AD (3.3–4.3×SE), +0.016 to +0.021 on VisA (2.7–5.5×SE) and +0.044 to +0.049 on NEU-DET (3.3–5.6×SE, every class), where every image is a texture judged better over a neighbourhood. They cost two extra memory searches and no extra encoder pass, and leave pixel AUROC unchanged (VisA: −0.005 at one shot). Text fusion helps most at one shot (+0.016 MVTec, +0.018 VisA, $\geq$ 3.9×SE); text alone is far worse than memory alone. On NEU-DET text *hurts* (−0.040 to −0.054, 3.4–4.0×SE): there the "normal" class is itself a defect type, so "a photo of a crazing with a defect" asks the wrong question. Fused per patch in the 512-dim space, text also hurts (1-shot image AUROC 0.820 at $w = 0$ falling to 0.646 at $w = 1$); fused per image on real products, it helps.

Table 3: Few-shot PatchCore given exactly our support photos, splits and test images (WideResNet-50, layers 2+3, no coreset subsampling), and the paired delta of ours minus PatchCore at 10 shots, with its multiple of SE. (`patchcore_vs_*.json`.)

	1-shot	5-shot	10-shot
PatchCore MVTec img/pix	.842/.938	.899/.958	.926/.965
PatchCore VisA img/pix	.793/.945	.848/.964	.865/.969
	MVTec img	VisA img	MVTec pix
ours B/16	−.022 (1.7)	−.034 (3.3)	−.029 (4.7)
ours B/16+win	−.005 (0.5)	−.015 (1.7)	−.027 (6.4)
ours L/14	+.007 (0.5)	−.022 (1.9)	−.020 (2.9)

Against PatchCore with the same photos. PatchCore is normally fitted on hundreds of images; here it receives exactly our support photos, test images, masks and metric code (Table 3). At detection the two are statistically level once our windows are on: no image-AUROC difference reaches 2×SE on either dataset, and L/14 leads at one shot on MVTec AD (+0.042, 2.0×SE). Plain single-pass B/16 trails on VisA (−0.034, 3.3×SE), which windows close. At localisation PatchCore wins by 0.02–0.03 pixel AUROC (3–6×SE); its 28×28 feature map is twice as fine as B/16's 14×14, and only L/14's 16×16 draws level on VisA. Per category the two have different strengths: ours on fine, textured defects (`screw` +0.120, `grid` +0.087, VisA `capsules` +0.050), PatchCore where a defect is a part out of place (`transistor` −0.173, VisA `pcb3` −0.115). Per image the two cost about the same on this laptop: PatchCore with 10 photos runs at 93 ms and 0.6 GB, our B/16 at 88 ms, our B/32 INT8 at 33 ms (Section 5.2). The case for this detector is therefore not raw accuracy or speed but what PatchCore lacks: products added in seconds without touching the others, typed descriptions, and one backbone shared by images and text.

Against WinCLIP. WinCLIP+ reports 93.1 / 94.4 / 95.2 image AUROC on MVTec AD and 83.8 / 84.6 / 87.3 on VisA at 1 / 2 / 4 shots, means over five seeds

Table 4: INT8 against FP32: 10-shot image AUROC, ONNX latency (batch 1, 6 threads) and model size. (`int8_*.json`, `int8_latency.json`.)

	MVTec	VisA	ms/img	MB
B/32 FP32	.904	.779	39–41	352
B/32 INT8	.891	.765	27–33	89
B/16 FP32	.904	.831	131–133	345
B/16 INT8	.885	.819	71–72	88

([5], Table 1), with a ViT-B/16+ backbone at 240 px, LAION-400M weights and one encoder pass per window. Our single-pass B/16 system reaches 0.881 (MVTec AD) and 0.815 (VisA) at one shot. We did not reproduce WinCLIP; across different backbones, weights and shot counts the comparison is indicative only.

INT8 costs little and changes nothing structural. Dynamic INT8 makes every graph 4× smaller and B/32 1.3× / B/16 1.8× faster (Table 4). It costs 0.005–0.02 image AUROC across both datasets and backbones. Which backbone loses more is dataset-dependent (B/16 on MVTec AD, neither clearly on VisA), and in no case does quantisation change which backbone should be chosen.

5.1 Adding a product

Twelve MVTec categories form a deployed memory; three held-out categories (fixed in advance as the alphabetically last) are appended from 5 or 10 photos. For every seed and shot count: (i) the twelve existing products score *bit-identically* before and after; (ii) each appended product scores bit-identically to a memory built with all fifteen from the start (`incremental_holdout_*.json`). Appending a product end to end (reading the photos, encoding, storing) takes 2.0–2.3 s from 5 photos and 3.3–3.7 s from 10 on the idle laptop, almost all of it encoding, and 3.0 MB (B/16, 5 shots); the memory work itself is 1.4–1.9 ms per product.

5.2 Cost

Burst latency (torch FP32, batch 1) is 37 ms for B/32 (3.0 GFLOPs), 88 ms for B/16 (11.3) and 397 ms for L/14 (52.0); the memory search adds 0.3–2.9 ms, about 1%, so extra shots are effectively free (Fig. 1). Peak RAM is 1.5 GB for B/32 and B/16 and 3.7 GB for L/14 in torch, which also loads the text tower; the ONNX visual tower needs 0.64–0.67 GB in FP32 and 0.38–0.39 GB in INT8. Under ten minutes of continuous inference the laptop steps down its clock: B/32 holds 40 ms for four minutes, then settles at about 230 ms, 5.8× slower; B/16 and L/14 run at 630–680 ms and about 2 s once throttled. Capacity should be planned on the sustained figure.

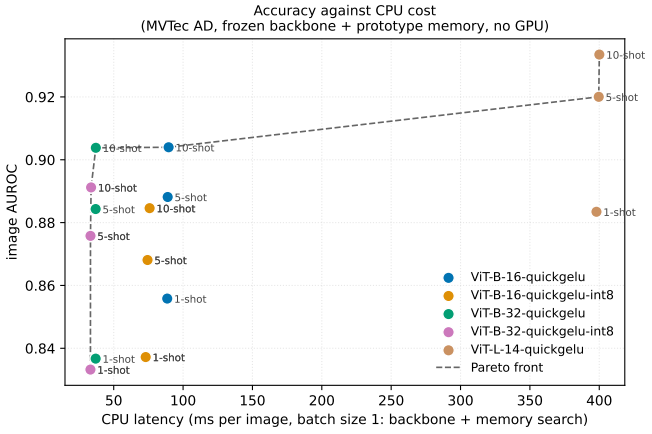


Figure 1: MVTec AD image AUROC against CPU latency per image (backbone plus memory search, batch 1, 6 threads), torch FP32 and ONNX INT8, 1/5/10 shots. The dashed line is the Pareto front. (`figures.json`.)

At 14 threads, timings on this hybrid CPU were unusable (the same graph measured 36 and 526 ms in consecutive runs); at 6 threads repeats agree within 20%. Under the same harness, PatchCore given our 10 photos runs at 93 ms (0.6 GB); fitted on all 209 `bottle` images it needs a 17.5-minute fit and runs at 170 ms with a 131 MB memory bank (`patchcore_cost_i7_*.json`). Assuming 45 W and Rs 8/kWh, 10,000 parts a day for 300 days costs Rs 9.80 a year with B/32 INT8 and Rs 40 with B/16 FP32; even throttled, the laptop is busy at most 1.8 hours a day with B/16 (5.6 with L/14). The study's benchmarks used at least 6.8 CPU-hours and no GPU; hardware spend was Rs 0 (`cost_feasibility.json`).

5.3 Verification

Each headline number was checked beyond the run that produced it. Re-extracting every feature from raw images reproduces all 135 MVTec AD (B/16) and 108 VisA (B/32) result rows to within 0.0002. NEU-DET was produced on one member's machine and reproduced on the other's: the split manifests regenerate byte-identically, and every per-class AUROC matches to within 0.0005. A fresh clone installed into a new environment from the pinned requirements passes all 193 tests. The architecture was frozen before the final experiments, and a test fails if any frozen model file changes.

6 Limitations

The datasets are public benchmarks under controlled capture, not a shop floor. Thin defects remain below the patch resolution (`screw`: 0.792 at 10 shots even with L/14), and PatchCore's finer feature map localises better. Image scores still confuse tiny defects with normal variation on VisA (0.855 best) while maps localise well

(pixel $\geq$0.95). With only three photos, the leave-one-out threshold the application suggests is too cautious and can miss defects; five photos or a hand-set threshold avoid this. We did not reproduce WinCLIP or AnomalyCLIP, test static INT8, or evaluate MPDD.

7 Conclusion

A frozen CLIP encoder with an append-only memory inspects parts from ten good photos at 0.925–0.933 image AUROC on MVTec AD on a laptop CPU, level with PatchCore given the same photos, and adding a product changes nothing about the products already in place, a property we verify exactly rather than approximately. The cost picture is more useful than any single latency: the cheapest backbone is enough when defects are large but not when they are smaller than its patches, a rule that held on all three datasets; INT8 trims 0.005–0.02 AUROC for a 1.3–1.8$\times$ speed-up without changing that choice; and a laptop that runs at 37 ms per image when cool settles at five to six times that under continuous use. Even at the throttled rate, a laptop the workshop already owns inspects 10,000 parts a day with hours to spare, for tens of rupees of electricity a year. The open questions are the ones public benchmarks cannot answer: defects thinner than a patch, and the gap between benchmark photographs and a shop floor.

Reproducibility. Code, committed split manifests, result records and one command per table (`python scripts/reproduce.py --list`): `https://github.com/Mounika-Reddy-0802/fewshot-msme-inspect`.

References

[1] Samet Akcay, Dick Ameln, Ashwin Vaidya, Barath Lakshmanan, Nilesh Ahuja, and Utku Genc. Anomalib: A deep learning library for anomaly detection. In *IEEE International Conference on Image Processing (ICIP)*, 2022.

[2] Paul Bergmann, Michael Fauser, David Sattlegger, and Carsten Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.

[3] Mehdi Cherti, Romain Beaumont, Ross Wightman, Mitchell Wortsman, Gabriel Ilharco, Cade Gordon, Christoph Schuhmann, Ludwig Schmidt, and Jenia Jitsev. Reproducible scaling laws for contrastive language-image learning. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[4] Thomas Defard, Aleksandr Setkov, Angelique Loesch, and Romaric Audigier. PaDiM: a patch distribution modeling framework for anomaly detection and localization. In *ICPR International Workshops and Challenges*, 2021.

[5] Jongheon Jeong, Yang Zou, Taewan Kim, Dongqing Zhang, Avinash Ravichandran, and Onkar Dabeer. WinCLIP: Zero-/few-shot anomaly classification and segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[6] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, 2021.

[7] Karsten Roth, Latha Pemula, Joaquin Zepeda, Bernhard Schölkopf, Thomas Brox, and Peter Gehler. Towards total recall in industrial anomaly detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

[8] Kechen Song and Yunhui Yan. A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects. *Applied Surface Science*, 285:858–864, 2013.

[9] Renrui Zhang, Wei Zhang, Rongyao Fang, Peng Gao, Kunchang Li, Jifeng Dai, Yu Qiao, and Hongsheng Li. Tip-Adapter: Training-free adaption of CLIP for few-shot classification. In *European Conference on Computer Vision (ECCV)*, 2022.

[10] Qihang Zhou, Guansong Pang, Yu Tian, Shibo He, and Jiming Chen. AnomalyCLIP: Object-agnostic prompt learning for zero-shot anomaly detection. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Yang Zou, Jongheon Jeong, Latha Pemula, Dongqing Zhang, and Onkar Dabeer. SPot-the-Difference self-supervised pre-training for anomaly detection and segmentation. In *European Conference on Computer Vision (ECCV)*, 2022.